WEBSOCKETS

LERNZIELE

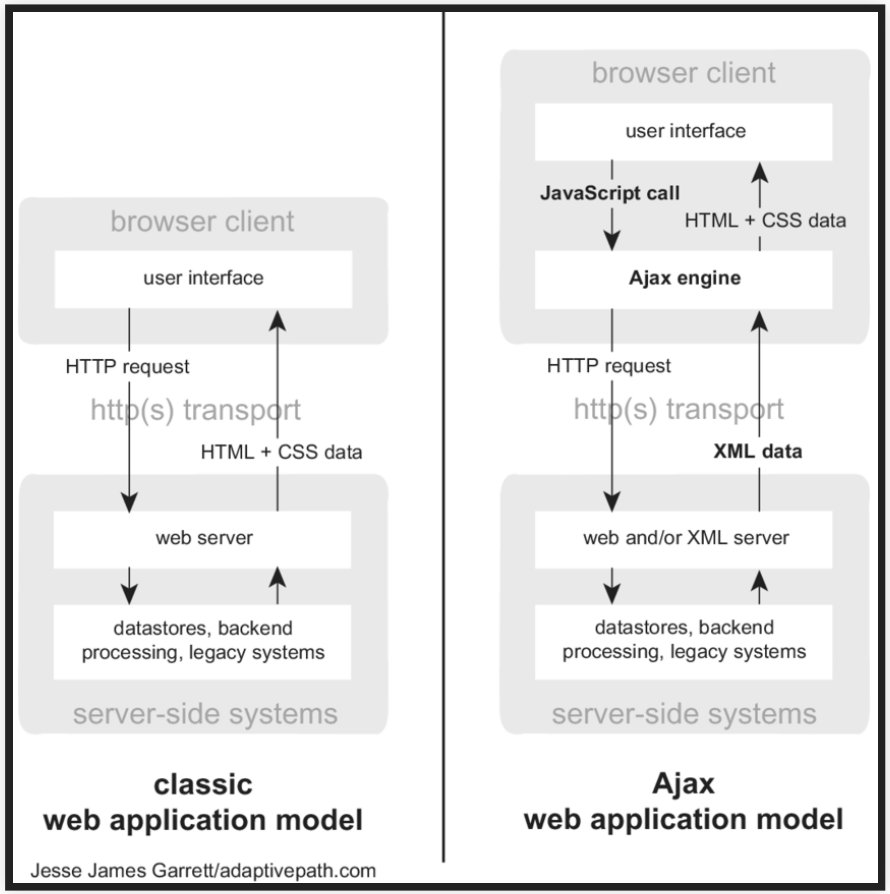
- Unterschied zwischen TCP/IP und Websockets sowie den Zusammenhang zwischen HTTP und Websockets erklären können
- Grundlegende Implementierung zur Verarbeitung von Verbindungen und Versenden von Nachrichten in einem Server bei der Verwendung von Websockets realisieren können
- Kommunikation mit einem Websocket Server im Client (z.B. JavaScript) realisieren können

GRUNDLAGEN

Entwicklung des World Wide Webs

- Ausgehend von 1989 durch Tim Berners-Lee als Software Engineer am Europäische Organisation für Kernforschung (CERN) mit einem "Web" aus Hypertext-Dokumenten
- Grundlage hierfür sind HTTP, URIs und HTML
- Seit 1997 ist HTTP/1.1 weit verbreitet, aktuell etabliert sich HTTP/2
- 1995 über Brendan Eich bei Netscape kam JavaScript als Möglichkeit zur Client-seitigen Programmierung hinzu
- JavaScript wurde insbesondere durch AJAX (Asynchronous JavaScript and XML) ca. 2005 in die Lage versetzt zusammen mit anderen Technologien als Grundlage für dynamische Webseiten genutzt zu werden

Klassische Webseiten vs. AJAX



Quelle: Jesse James Garrett, Ajax: A New Approach to Web Applications (https://designftw.mit.edu/lectures/apis/ajax_adaptive_path.pdf)

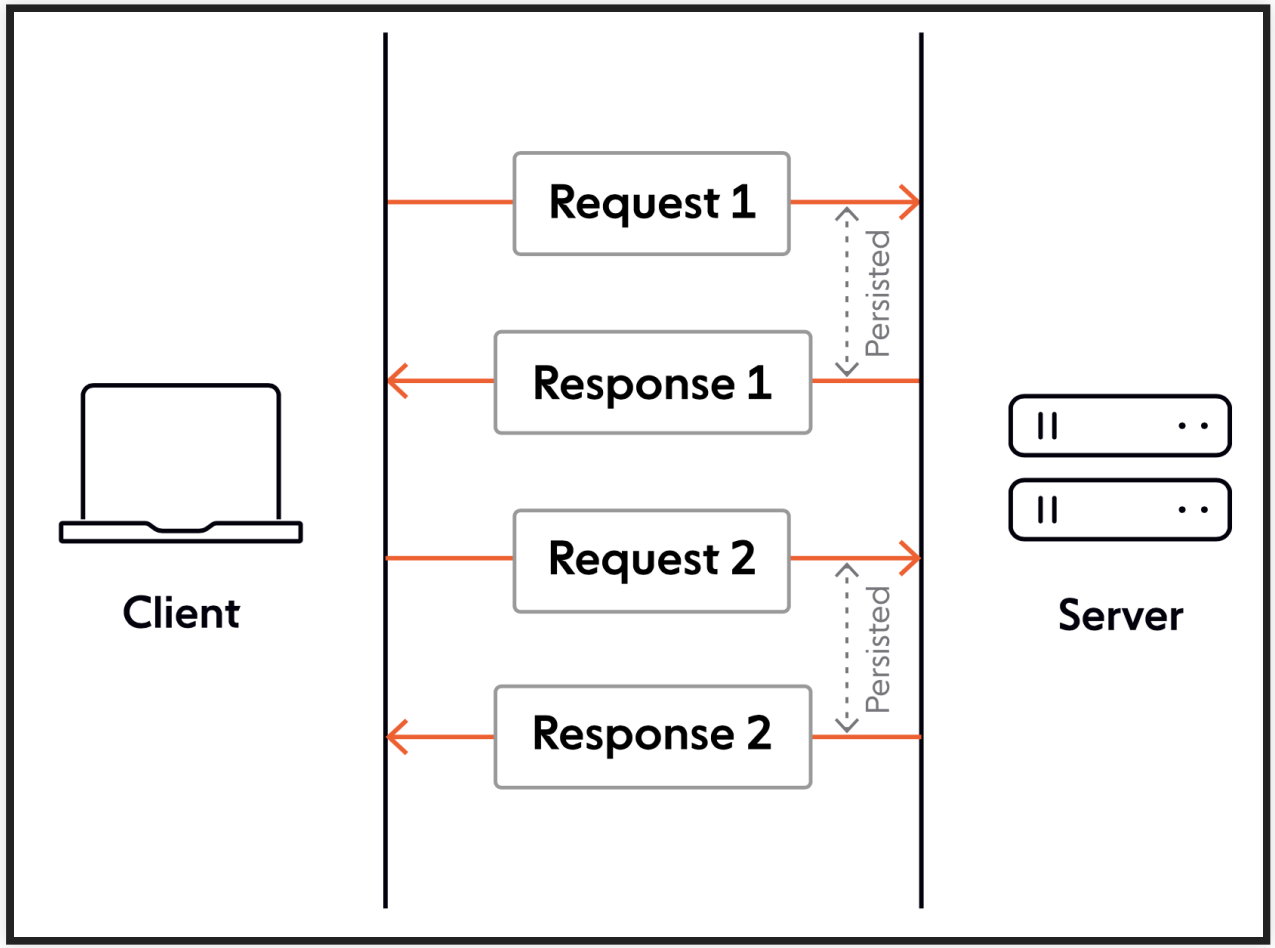
Wie kann der Server den Client über Zustandsänderungen informieren?

Klassische Lösung bei HTTP wäre Polling, wiederholtes Anfragen in einer festen Frequenz, da mittels JavaScript nur HTTP-basierte Anfragen möglich sind.

Comet

- Comet wurde 2006 ins Leben gerufen und ist ein Designmodell für Webanwendungen
- ermöglicht es Webserver, Daten an den Browser zu senden
- ähnlich wie AJAX ermöglicht Comet eine asynchrone Kommunikation
- Comet verwendet langlebige HTTP-Verbindungen, damit der Server Aktualisierungen immer dann bereitstellen kann, wenn sie verfügbar sind, ohne dass der Client sie explizit anfordert

Long Polling als Grundlage für Comet



Quelle: Alex Diaconu, The WebSocket Handbook

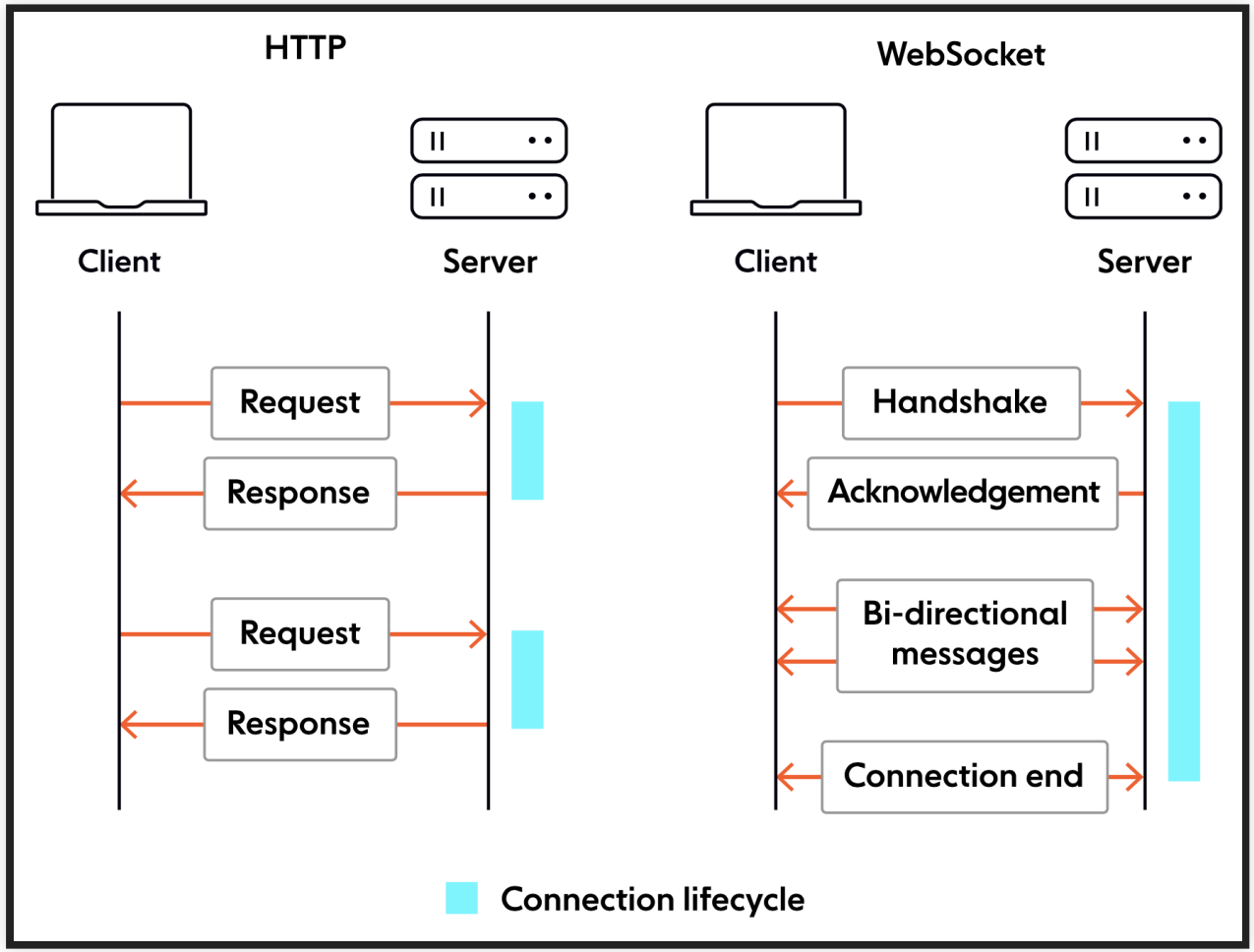
Einschränkungen von HTTP

- Begrenzte Skalierbarkeit, insbesondere Polling mit hohen Frequenzen belastet die Infrastruktur
- Unzuverlässig bei der Anordnung und Zustellung von Nachrichten
- Latenz
- Kein echtes bidirektionales Streaming

... und ein direkter TCP/IP Client Socket ist in JavaScript nicht möglich.

Quelle: Alex Diaconu, The WebSocket Handbook

WebSockets als Alternative



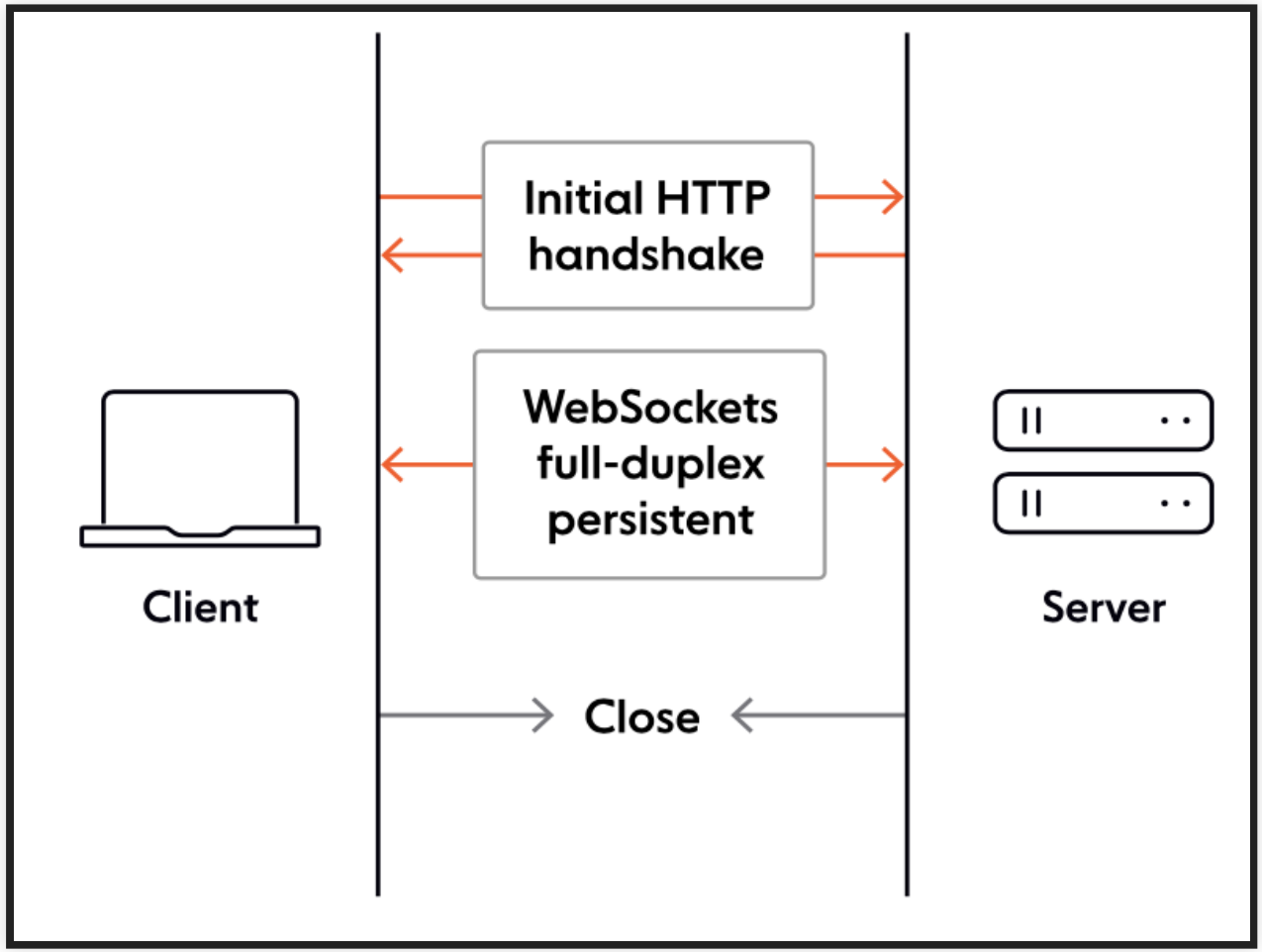
Quelle: Alex Diaconu, The WebSocket Handbook

Die WebSocket-Technologie umfasst zwei Kernbausteine:

- Das **WebSocket-Protokoll**: Ermöglicht die Kommunikation zwischen Clients und Servern über das Internet und unterstützt die Übertragung von Binärdaten und Textstrings
- Die **WebSocket-API**: Ermöglicht die Durchführung notwendiger Aktionen, wie die Verwaltung der WebSocket-Verbindung, das Senden und Empfangen von Nachrichten und das Abhören von Ereignissen, die vom Server ausgelöst werden

PROTOKOLL

Beschrieben in RFC 6455



Quelle: Alex Diaconu, The WebSocket Handbook

Aufbau einer URL

```
  wss://example.com:443/some/path/element?p1=A&p2=B
  \_/   \_________/ \_/\_________________/ \_______/
   |         |       |       |                |
Schema+     Host    Port    Pfad            Query
```

- Als Schema kann `ws` für unsichere Verbindungen und `wss` für sichere Verbindungen verwendet werden
- Host, Port, Pfad und Query entsprechen dem einer URL in HTTP(S)
- Default Port ist 80 für `ws` und 443 für `wss`

Quelle: Alex Diaconu, The WebSocket Handbook

Client Request

```
GET /chat HTTP/1.1
Host: server.example.com
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Key: dGhlIHNhbXBsZSBub25jZQ==
Sec-WebSocket-Protocol: chat, superchat
Sec-WebSocket-Version: 13
Origin: http://example.com
```

Der initiale HTTP-Handshake entspricht einer GET-Anfrage, ergänzt um ein Connection-Upgrade.

Quelle: RFC 6455

Server Response

```
HTTP/1.1 101 Switching Protocols
Upgrade: websocket
Connection: Upgrade
Sec-WebSocket-Accept: s3pPLMBiTxaQ9kYGzzhZRbK+xOo=
```

Als Antwort sollte eine Bestätigung mit dem Ziel das Protokoll zu wechseln kommen. Im Ergebnis wird nicht wie bei klassischen HTTP-Anfragen die Verbindung beendet.

Quelle: RFC 6455

Data Framing

```
 0                   1                   2                   3
 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1 2 3 4 5 6 7 8 9 0 1
+-+-+-+-+-------+-+-------------+-------------------------------+
|F|R|R|R| opcode|M| Payload len |    Extended payload length    |
|I|S|S|S|  (4)  |A|     (7)     |             (16/64)           |
|N|V|V|V|       |S|             |   (if payload len==126/127)   |
| |1|2|3|       |K|             |                               |
+-+-+-+-+-------+-+-------------+ - - - - - - - - - - - - - - - +
|     Extended payload length continued, if payload len == 127  |
+ - - - - - - - - - - - - - - - +-------------------------------+
|                               |Masking-key, if MASK set to 1  |
+-------------------------------+-------------------------------+
| Masking-key (continued)       |          Payload Data         |
+-------------------------------- - - - - - - - - - - - - - - - +
:                     Payload Data continued ...                :
+ - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - - +
|                     Payload Data continued ...                |
+---------------------------------------------------------------+
```

Quelle: RFC 6455

Inhalt des Data-Frames

- FIN bit - zeigt an, ob der Rahmen das letzte Fragment einer WebSocket-Nachricht ist
- RSV 1, 2, 3 - reserviert für WebSocket-Erweiterungen
- Opcode - bestimmt, wie die Nutzdaten zu interpretieren sind.
- Mask - gibt an, ob die Nutzdaten maskiert sind oder nicht.
- Masking key - Schlüssel, der zum Demaskieren der Nutzdaten verwendet wird
- (Extended) payload length - die Länge der Nutzdaten
- Payload data - Anwendungs- und Erweiterungsdaten

Quelle: Alex Diaconu, The WebSocket Handbook

- Nach einem erfolgreichen Opening Handshake können der Client und der Server die WebSocket-Verbindung zum Austausch von Nachrichten im Vollduplex-Modus nutzen
- Eine WebSocket-Nachricht besteht aus einem oder mehreren Frames
- Der WebSocket-Frame hat eine binäre Syntax und enthält mehrere Informationen

Sub-Protokolle

Es ist möglich anwendungsspezifische (Sub-) Protokolle beim Handshake anzugeben, welche vom Server mit der Bestätigung ausgewählt werden können.

```
Sec-WebSocket-Protocol: amqp, v12.stomp
```

Quelle: Alex Diaconu, The WebSocket Handbook

SERVER (NODEJS)

Ein WebSocket-Server kann in jeder serverseitigen Programmiersprache geschrieben werden, die Berkeley-Sockets unterstützt.

Der Server lauscht auf eingehende WebSocket-Verbindungen über einen Standard-TCP-Socket. Sobald das Opening Handshake ausgehandelt ist, muss der Server in der Lage sein, WebSocket-Nachrichten zu senden, zu empfangen und zu verarbeiten.

Für die Beispiele ist NodeJS und NPM notwendig

Beispiel in NodeJS

```
const WebSocketServer = require('ws').WebSocketServer;

const wss = new WebSocketServer({ port: 12345 });

wss.on('connection', function connection(ws) {
  ws.on('message', function message(data) {
    console.log('received: %s', data);
  });

  ws.send('something');
});
```

- Mit dem Connection-Event werden neue Verbindungen verarbeitet
- Mit dem Message-Event je Connection werden eingehende Nachrichten verarbeitet
- Mittels `send`-Aufruf können Nachrichten an den Client gesendet werden

Hierfür ist über NPM die Dependency `ws` zu installieren.

Projekt erstellen und starten

```
npm init -y
npm install ws --save
node server.js
```

- Erzeugt ein neues NPM-Projekt
- Installiert die Dependency
- Startet den Server, hierfür muss der Snippet in server.js gespeichert sein

Nutzen Sie das bereitgestellte Projekt-Archiv nodejs-websocket.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

CLIENT

Grundlage für den Client ist in JavaScript die Klasse `WebSocket`.

```
let socket = new WebSocket("ws://localhost:12345");
```

Das Beispiel erzeugt einen WebSocket-Client und verbindet sich mit dem Host `localhost` auf Port 12345.

Ereignisse verarbeiten

```
let socket = new WebSocket("ws://localhost:12345");
socket.addEventListener('open', function() { ... })
socket.addEventListener('message', function() { ... });
socket.addEventListener('close', function() { ... });
socket.addEventListener('error', function() { ... });
```

- Wird aufgerufen, wenn die Verbindung erfolgreich etabliert werden konnte
- Wird immer aufgerufen, wenn eine Nachricht empfangen wurde (vom Server)
- Wird immer aufgerufen, wenn die Verbindung geschlossen wurde
- Wird im Fall von Fehlern aufgerufen

Im NodeJS-Beispiel finden Sie eine `index.html`, diese kann als Beispiel genutzt werden.

Nutzen Sie das bereitgestellte Projekt-Archiv nodejs-websocket.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

SERVER (JAVA)

WebSocket-Server lassen sich mit verschiedenen Frameworks umsetzen, Ziel ist hierbei immer der bidirektionale Austausch von Nachrichten

Folgendes Beispiel zeigt eine einfache Implementierung mittels Java, Jarkata und einem Glassfish-Server.

```java
@ServerEndpoint("/{topic}")
public class WebSocketServer {
    @OnOpen
    public void onOpen(Session session, @PathParam("topic") String topic)
        throws IOException {... }
    @OnMessage
    public void onMessage(String message) { ... }
    @OnError
    public void onError(Throwable t) { ... }
    @OnClose
    public void onClose(Session session) { ... }
}
```

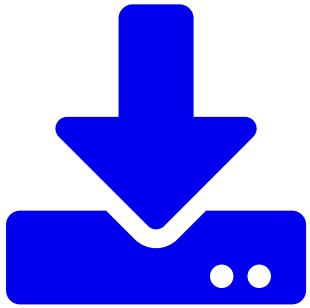
Hierfür werden Annotationen genutzt, um Eigenschaften einer Klasse für die Verarbeitung von WebSocket-Verbindungen zu annotieren.

Mittels `@ServerEndpoint` wird dabei die Klasse als WebSocketServer markiert.

Über `@OnOpen`, `@OnMessage`, `@OnError` und `@OnClose` lassen sich Methoden markieren, die die Ereignisse verarbeiten.

Starten des Glasfish-Servers

```
Server server = new Server("localhost", 12345, "/",
    new HashMap<>(), WebSocketServer.class);
try {
    server.start();
    System.out.println("--- server is running");
    System.out.println("--- press any key to stop the server");
    BufferedReader bufferRead = new BufferedReader(
        new InputStreamReader(System.in));
    bufferRead.readLine();
} catch (Exception e) {
    e.printStackTrace();
} finally {
    server.stop();
}
```

- Erzeugen des Servers
- Starten des Servers
- Java-Prozess blockieren, damit der Server weiterhin ausgeführt wird.
- Stoppen des Servers

Nutzen Sie das bereitgestellte Projekt-Archiv java-websocket.zip, entpacken Sie dieses und importieren Sie den resultierenden Ordner es als "Existing Maven Project".